1. The Core Principle: Brightness Constancy

The fundamental assumption of this code is that **if an object moves, the color/brightness of its pixels stays the same.** When CJ walks or the car moves, a specific gray value (e.g., the gray of a tire) moves from one (x, y) coordinate to another in the next frame. The algorithm looks for where that specific gray value went.

---

2. Understanding the "Gradients" (The Inputs)

Before calculating movement, the code analyzes the "Gradients." Think of a gradient as a **measurement of change.**

- **Horizontal Gradient (I_x):** This measures how much the brightness changes as you move from left to right. It highlights vertical edges (like the side of a building).
- **Vertical Gradient (I_y):** This measures how much the brightness changes as you move from top to bottom. It highlights horizontal edges (like the roof of the car).
- **Temporal Gradient (I_t):** This is the **Change Over Time**. The code subtracts the old frame from the new frame. If a pixel was black and is now white, the temporal gradient is high, signaling that "something moved here."

---

3. The "Window" Logic (Spatial Coherence)

The code doesn't look at just one pixel at a time because one pixel doesn't provide enough information to determine direction. Instead:

- It looks at a **Local Neighborhood** (a 15×15 pixel box).
- It assumes that every pixel inside that little box is moving at the exact same speed and in the same direction.

- By looking at 225 pixels (15x15) at once, the algorithm has enough "votes" to accurately guess the movement.

---

4. Solving the Movement (The "Least Squares" Solver)

The heart of the code is a mathematical "balancing act" called the **Least Squares Method**.

1. **The Goal:** Find two numbers—the **Horizontal Velocity (u)** and the **Vertical Velocity (v)**.
2. **The Challenge:** We have 225 pixels in our window, each providing a slightly different "clue" about the movement.
3. **The Solution:** The code builds a system of equations. Since the clues might not perfectly align due to noise or shadows, it calculates the **Line of Best Fit**. It finds the u and v values that minimize the total error across all 225 pixels.
4. **Invertibility Check:** If the area is perfectly flat (like a plain white wall), there are no "clues" (gradients are zero). The code detects this by checking the **Determinant**. If it's too low, it admits it's lost and stops tracking that point.

---

5. Visualization: The "Motion Trail" Logic

To create the "Light Beam" effect:

- **Vector Calculation:** The code takes the old position and adds the calculated velocity (u, v) to find the new position.
- **The Mask:** It draws a line connecting the **Old Dot** to the **New Dot**.
- **Persistence:** Because we never erase the "Mask" image, these lines stack up frame-by-frame, creating a continuous trail that reveals the path of the car or the person.

6. Summary of Variables in Plain English

Variable	Mathematical Term	Plain English Meaning
Ix / Iy	Spatial Derivatives	Where are the edges in the image?
It	Temporal Derivative	What part of the image changed since the last frame?
ATA	Covariance Matrix	How strong and reliable are the edges in this local box?
d (u, v)	Displacement Vector	How many pixels left/right and up/down did the object move?
p0 / p1	Point Coordinates	The "Before" and "After" locations of the object.